

EventVLA 深读：把“何时该记住”变成一个可学习的预测问题

论文：EventVLA: Event-Driven Visual Evidence Memory for Long-Horizon Vision-Language-Action Policies

作者：Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, Wengang Zhou, Yao Mu, Tai Wang（中科大 / 上海 AI Lab / 华为 / 港大 / 清华 / 北大 / 上交）

arXiv: 2606.20092 (2026-06-18) · 代码与数据: InternRobotics/EventVLA

仓库内文件: 英文 PDF · 中文翻译 PDF · 英文版解读

0. 一句话

EventVLA 让 VLA 策略自己预测“接下来这 50 步里哪几帧将来会用得上”，只把这几帧原图存进一个容量为 5 的缓冲区，再和初始帧、近期帧一起送回视觉编码器。这样做在作者新建的 RoboTwin-MeM 基准上把平均成功率从 18.0%（只用固定锚帧）提到 75.2%，在 RMBench 上只用固定锚帧就拿到 67.8%（此前最好的 Mem-0 是 42.0%），真机四个任务成功率 90 / 60 / 90 / 75%。

1. 要解决的问题

标准 VLA 是马尔可夫的： $a_t = \pi(o_t, l)$ ，默认所有任务相关信息一直看得见。真实操作里信息经常只出现一瞬间：掀开盖子看一眼颜色再盖上、读一张随机数字卡、看别人用木棍点一遍顺序。这些证据出现后就消失，而动作要在几百步之后才依赖它。作者把这类任务叫“非马尔可夫操作”，并把已有的记忆增强路线分成三类，各有一个具体的失败方式：

路线	代表	失败方式
双系统（高层 VLM 管记忆 + 低层策略执行）	MemER、Mem-0、MEM (π_{MEM})	延迟高、误差沿层级传播
循环 / 隐状态压缩	AVA-VLA、Recurrent Memory Transformer	信息瓶颈，细粒度视觉细节被丢掉
帧缓冲（保留历史原图）	MemoryVLA、CronusVLA、ContextVLA、LoLA	无选择地堆帧，冗余淹没稀疏关键证据，计算开销大

论文提出的核心问题只有一句：**VLA 究竟应该在什么时候、保留什么视觉证据，才能既完成任务又不撑爆算力？**

2. 方法

2.1 记忆的结构：锚帧 + 事件帧

把策略改写成 $a_t = \pi(o_t, M_{\{t-1\}}, l)$ ，其中记忆 $M_t = A_t \cup E_t$ ：

- **基础视觉锚帧** $A_t = \{o_0\} \cup \{o_{t-K}, \dots, o_{t-1}\}$ ：初始帧提供不变的全局布局（东西一开始在哪），短期滑窗提供运动与任务进度线索。这部分是规则化的、不需学习。仿真里用 o_0, o_{t-30}, o_{t-15} ；真机用 $o_0, o_{t-60}, o_{t-40}, o_{t-20}$ 。
- **事件关键帧** E_t ：由 KEM 模块动态写入，容量上限 $N_{\max} = 5$ ，FIFO 淘汰。

推理时三部分按时间顺序拼成一个图像序列 $I_{\text{input}} = \text{concat}([A_t, E_{t-1}, o_t])$ ，直接进 VLM 的视觉编码器，让自注意力自己在稀疏历史帧之间建立时间关联。没有独立的记忆读取模块——“读”就是普通的多帧注意力。

2.2 KEM：预测未来关键帧的概率

KEM 是一个挂在动作头旁边的轻量 MLP 头。它的输入是 VLA 自回归 Transformer 最后一层的隐状态 $h_t \in \mathbb{R}^{H \times d}$ （ $H = 50$ 是动作块长度），输出一个长度为 H 的概率向量：

$$\hat{p}_t = \sigma(\text{KEM_mlp}(h_t)) = [\hat{p}_t^1, \dots, \hat{p}_t^H] \in [0, 1]^H$$

$\hat{p}_t^i$ 表示“未来第 i 步会是任务关键帧”的概率。之所以按整块预测而不是逐步做二分类，是因为关键事件常常在一个动作块执行到一半时出现又消失——逐步分类器根本来不及。共享隐状态是关键设计： h_t 同时编码了观测和动作查询，所以 KEM 天然知道策略接下来要做什么，能“预约”一次记忆写入。

写入规则： $\hat{p}_t^i \geq \tau_{\text{commit}}$ （0.55）时，把 $t+i$ 时刻的原图写进 E_t 。为了防止同一事件的相邻帧把缓冲区灌满，在线还有两步后处理：

1. **一维 NMS**：在半径 $w = 8$ 的滑窗内只保留局部峰值；
2. **冷却期**：距离上一次写入不足 $C = 10$ 步的候选被丢弃。

2.3 训练：自动标注 + 软标签 + 师生课程

- **标签来源**：用 Qwen3-VL-235B-A22B（8 卡 A800，vLLM）离线看演示视频，每回合均匀采 128 帧、拼多视角，few-shot 提示后直接输出 JSON 格式的关键帧步数。仿真里对照物理引擎的真值，平均时间误差小于 10 步；真机对照人工标注误差在 50 步以内（回合长度 1500–2000 步）。
- **软标签**：物理事件在时间上是模糊的，硬 0/1 标签让预测头不稳定。作者用升余弦核把标签平滑到半径 R 内： $y_t^i = 0.5(1 + \cos(\pi|t+i-t^*|/R))$ 。
- **损失**： $L = L_{\text{action}} + \lambda L_{\text{kem}}$ ， L_{kem} 是序列平均的 BCE， $\lambda = 0.1$ 。
- **课程**：训练早期缓冲区用真值关键帧构造（teacher forcing），概率 α 从 1 线性衰减到 0，逐步换成模型自己的阈值预测。这一步是为了消除“训练用真值、测试用预测”的分布偏移。

2.4 骨干与超参

设置	RM Bench / RoboTwin-MeM	真机
基座	QwenOFT (Qwen3-VL-4B-Instruct, StarVLA 代码库)	$\pi_{0.5}$ (PaliGemma)
动作头	OFT, $H = 50$, 动作维 14	flow 头, $H = 50$, 动作维 32
训练步数	80k, VLM 学习率 $1e-5$, 新模块 $1e-4$	60k, $5e-5$ 余弦衰减, 全局 batch 32

设置	RMBench / RoboTwin-MeM	真机
图像分辨率	224×224	224×224
KEM	$\tau=0.55$, $N_{\max}=5$, $w=8$, $C=10$, $\lambda=0.1$	同左

3. RoboTwin-MeM 基准

作者认为 RMBench 和 RoboMME 这类基准大多能靠”初始帧 + 近期帧”解决，真正只在交互中间短暂出现的证据没有被单独测过。RoboTwin-MeM 建在 RoboTwin 2.0 (SAPIEN) 上，8 个双臂任务，每个任务用一个参数 $n \in [1, 5]$ 标明”必须记住几个中间关键帧”：

任务	n	平均步数	考什么
Rearrange Blocks Hard	1	879	记住搬走的是哪块
Put Back Block Hard	2	1468	记住两块分别去过哪个外侧垫
Pick Objects in Order / Pick the Unhidden Block	3	1124 / 699	掀盖看内容，盖回后按顺序/排除法取
Cover Blocks Hard / Find Seal Stamp / Reproduce Route	4	1544 / 1338 / 1417	多次掀盖记颜色；找印章并放回原盖下；看一遍演示后复现路线（上下文学习）
Press Button Keyframe	2–5	430	读两张数字卡，按对应次数按按钮（计数）

三类能力被拆开测：瞬态证据记忆（掀盖类）、序列/计数（按钮类）、上下文模仿（Reproduce Route）。每个任务 50 条演示，附带与每个动作-观测对对齐的细粒度语言标注。

4. 实验结果

4.1 RMBench：只用锚帧就够了

方法	平均成功率
$\pi_{0.5}$ / X-VLA / QwenOFT（无记忆）	10.4 / 9.8 / 5.6
MemER / Mem-0（双系统）	8.7 / 42.0
MemoryVLA (OpenVLA) / MemoryVLA (QwenOFT)	19.4 / 41.7
EventVLA 去掉初始帧 / 去掉短期窗	33.7 / 23.8
EventVLA（仅视觉锚帧）	67.8

RMBench 的 9 个任务依赖持久的空间布局和固定的动作风格，所以作者只部署了锚帧版本。两个消融说明初始帧和短期窗都不可少：去掉任一个就分别跌到 33.7% 和 23.8%。逐任务看，Rearrange Blocks 96%、Put Back Block 95%、Swap Blocks 96%、Cover Blocks 97%，但 Observe and Pick Up 只有 21%，Press Button 只有 3%——计数类任务锚帧解决不了。

4.2 RoboTwin-MeM：KEM 带来的是质变

方法	n=1	n=2	n=3	n=3	n=4	n=4	n=4	n=5	平均
$\pi_{0.5}$	20	19	1	14	0	8	0	0	7.8
MemER	32	4	12	2	0	26	3	5	10.5
MemoryVLA (QwenOFT)	39	0	1	9	1	11	0	25	10.8
EventVLA 仅锚帧	62	13	5	20	0	26	0	18	18.0
EventVLA 锚帧 + KEM	62	93	90	54	94	63	98	48	75.2

（列顺序：Rearrange Blocks Hard、Put Back Block Hard、Pick Objects in Order、Pick the Unhidden Block、Cover Blocks Hard、Find Seal Stamp、Reproduce Route、Press Button Keyframe。）

从 18.0% 到 75.2% 的跳变全部来自 KEM。n=1 的任务锚帧已经够用（62% 不变），n≥2 的任务几乎全靠事件帧。Mem-0 在这套基准上是 0%。

4.3 消融：每个设计选择值多少

变体	平均	说明
完整 EventVLA	75.2	—
隐式记忆库（把关键帧压成一个 latent 向量）	24.9	多事件挤进一个向量，信息瓶颈
硬标签	48.8	预测头不稳定，漏写
去掉 NMS	53.4	同一事件的相邻帧灌满缓冲区，早期证据被 FIFO 挤出
N_max = 2	32.0	容量不够
动作块 30 / 15	31.1 / 13.6	前瞻窗口缩短，KEM 来不及预约写入

“隐式记忆库 24.9 vs 原图 75.2”是全文最有分量的一个数字：它直接反驳了”把历史压成 latent 就够了”的默认做法——至少在需要同时记住 3 到 5 个视觉细节的任务上不够。

4.4 不损伤马尔可夫任务

RoboTwin 2.0 标准任务上，EventVLA 相对 QwenOFT 基座 Easy 80.0→83.8%、Hard 78.0→81.6%，比 $\pi_{0.5}$ （82.7 / 76.8）也略高。

4.5 真机：ARX ACONE 双臂

四个任务各 20 次：Find Block Easy / Hard（藏块位置只短暂可见）、Pick-X-Times（读随机数字后计数）、Pick in Order（木棍点一遍顺序）。

方法	Find Block Easy	Find Block Hard	Pick-X-Times	Pick in Order
$\pi_{0.5}$	0–10%	0–10%	0–10%	0–10%
π_{MEM} （复现）	50	—	30	40

方法	Find Block Easy	Find Block Hard	Pick-X-Times	Pick in Order
EventVLA	90	60	90	75

4.6 代价：推理速度降到三分之一

方法	平均延迟 (s/chunk)	吞吐 (chunks/s)
QwenOFT	0.36	2.91
EventVLA 仅锚帧	0.96	1.07
EventVLA 锚帧 + KEM	1.09	0.94

多帧输入让视觉 token 数量翻了几倍，吞吐从 2.91 Hz 降到 0.94 Hz。作者的辩护是 VLA 通常作为高层规划器搭配高频底层控制器，1 Hz 够用。

5. 判读

5.1 真正的贡献在哪

- 1. 把”何时写入”从规则变成预测。之前的方法要么每帧都写（帧缓冲），要么按固定间隔采样，要么交给一个外挂 VLM 决定。KEM 用策略自己的隐状态预测未来 50 步的写入概率，相当于让策略”预约”记忆。这和动作块预测共享同一个表征，几乎不增加参数。
- 2. 用一个干净的对照证明了原图优于 latent。75.2 vs 24.9 的差距是在同一框架、同一数据、只换记忆表征下测出来的，比跨论文比较可信得多。
- 3. 用 n 参数化记忆需求。RoboTwin-MeM 把”要记住几件事”变成任务的显式属性，可以画出成功率随 n 的衰减曲线，这是以前的记忆基准做不到的诊断能力。

5.2 需要留意的地方

- 1. KEM 没有在 RMBench 上测。RMBench 的 67.8% 是纯锚帧版本，KEM 的收益只在作者自建的 RoboTwin-MeM 上体现。两个基准都出自 RoboTwin 系（RMBench 第一作者 Tianxing Chen 也是本文作者），设计取向天然贴合”锚帧 + 关键帧”的假设。RoboMME、MIKASA-Robo 上的表现没有报告。
- 2. KEM 与长动作块强耦合。动作块从 50 缩到 15 时成功率从 75.2% 掉到 13.6%，比去掉 KEM（18.0%）还低。这意味着 KEM 的前瞻能力是动作块长度的函数；对需要短块高频响应的任务（接触丰富、动态场景），这套机制未必能直接搬过去。
- 3. 容量 5 + FIFO 的天花板。作者在局限里明说超过 10 分钟、事件密集的任务会撑爆缓冲区并挤掉早期证据。RoboTwin-MeM 最难的 n=5 任务（Press Button Keyframe）也只有 48%，n=4 的 Find Seal Stamp 63%，成功率随 n 明显下降。
- 4. 标注依赖 235B 模型。8 卡 A800 跑 Qwen3-VL-235B 做离线标注，这个成本在论文里没有量化。真机上关键帧误差在 50 步以内，靠软标签的时间容忍度吃掉。
- 5. 三倍延迟。0.94 Hz 对桌面双臂任务够用，对移动操作或动态场景是实打实的代价。NativeMEM、TempoFit 这类工作正是在这个维度上做文章（每帧压成 1 个 token / 不加 token 只复用 KV）。

6. 真机每任务 20 次，没有置信区间； π _MEM 是作者复现版。

5.3 与 TRACE 的对照

两篇论文解决的是同一个问题（决策时刻证据已经消失），但在三个轴上取了相反的设计：

维度	EventVLA	TRACE
记忆内容	原始图像帧	512 维 latent 槽
写入触发	学到的“未来关键帧”概率 + NMS + 冷却	每步都写，但由轨迹签名路由到不同槽、门控更新
寻址方式	时间顺序拼接，靠注意力自己找	机器人状态轨迹的路径签名作为键
容量	5 帧 + 锚帧	$K = 4$ 或 6 槽，固定
与骨干的关系	端到端，进 VLM 视觉编码器	适配器外挂，骨干、动作头、损失都不改
推理开销	吞吐 $2.91 \rightarrow 0.94$ Hz	每步 +2.1 ms

两种设计对应两类信息：需要保留的是**高维视觉细节**（颜色、位置、多个物体）还是**低维分支变量**（是哪个来源、走哪条路）。EventVLA 的隐式记忆库消融（24.9%）说明前者压不进一个向量；TRACE 的成功说明后者放进几个槽就够。选哪种记忆，取决于下游要恢复的信息有多少比特。

6. 关联阅读

- **KEMO** (2606.23589, 同月)：也是事件驱动关键帧记忆，但事件检测用机器人运动学 + 视觉过滤的规则而不是学习的预测头；关键帧编码为 token 后经交叉注意力和门控残差融合进 VLA。真机相对 $\pi_{0.5}$ 任务成功率 +23.6%。两篇几乎同时出现，说明“事件驱动的稀疏写入”是 2026 年中的共识方向。
- **UniMem** (2608.22869)：单骨干里放一个事件分类器决定何时更新记忆、一个关键帧编码器做密集空间记忆，仿真 93.4% vs 固定间隔采样 68.2%。
- **MemoryWAM** (2606.20562)：世界动作模型里用“近期帧 + 事件边界锚帧 + gist token”三层记忆，与 EventVLA 的锚帧 + 事件帧结构同构。
- **MemER** (2510.20328)：双系统里由高层 VLM 选关键帧；本文把它当作双系统基线，RoboTwin-MeM 上 10.5%。
- **RMBench** (2603.01229) 与 **RoboMME** (2603.04639)：本文认为二者可被锚帧解决，RoboTwin-MeM 正是为了补“瞬态证据”这一空白。
- **Present but Not Remembered** (2607.03372)：探针实验发现冻结 VLA 里的历史帧信息基本是当前帧的冗余副本，建议记忆增强应注入“只属于过去”的信息——这正是稀疏事件帧在做的事。